

Laporan Praktikum Minggu Ke-6 Kontrol Cerdas

Minggu ke-6

Nama : Fatmawati Tia Pratama Putri
NIM : 224308056
Kelas : TKA 7C
Akun Github (Tautan) : <https://github.com/fatmawati25>

1. Judul Percobaan

Canny Edge Detection & Lane Detection with Instance Segmentation

2. Tujuan Percobaan

Tujuan dari percobaan pada praktikum kali ini, sebagai berikut:

- Memahami konsep **Canny Edge Detection** sebagai metode dasar deteksi tepi.
- Menggunakan **Instance Segmentation** untuk deteksi jalur rel kereta (Lane Detection).
- Menggunakan dataset **Rail Segmentation** dari Kaggle untuk eksperimen.
- Menggabungkan metode **Canny Edge Detection** dengan Instance Segmentation untuk meningkatkan deteksi jalur.

3. Landasan Teori

Canny edge detector adalah deteksi tepi operator yang menggunakan multi-tahap algoritma untuk mendeteksi berbagai tepi dalam gambar. Ini dikembangkan oleh John F. Canny pada tahun 1986. Canny juga menghasilkan *teori komputasi deteksi tepi* yang menjelaskan mengapa teknik ini bekerja. Proses ini terdiri dari beberapa tahapan utama, yaitu:

1. Noise Reduction (Reduksi Noise)

Sebelum mendeteksi tepi, gambar terlebih dahulu difilter menggunakan **Gaussian Blur** untuk mengurangi noise yang dapat menyebabkan deteksi tepi palsu.

2. Gradient Calculation (Perhitungan Gradien)

Setelah noise berkurang, setiap piksel dianalisis untuk menghitung **arah dan besar gradien** menggunakan operator seperti Sobel. Nilai gradien ini menunjukkan seberapa besar perubahan intensitas piksel, yang menandakan adanya tepi.

3. **Non-Maximum Suppression (Penekanan Non-Maksimum)**

Tahapan ini bertujuan untuk **menipiskan tepi** dengan menghapus piksel yang bukan merupakan bagian dari puncak gradien maksimum pada arah tepi.

4. **Double Threshold (Ambang Ganda)**

Dua nilai ambang digunakan untuk mengklasifikasikan piksel tepi menjadi **strong edge**, **weak edge**, atau **non-edge**.

5. **Edge Tracking by Hysteresis (Pelacakan Tepi dengan Histeresis)**

Pada tahap akhir, **weak edge** yang terhubung dengan **strong edge** dipertahankan, sementara yang tidak terhubung akan dihapus. Hasilnya adalah peta tepi yang jelas dan kontinu.

Lane Detection menggunakan **Instance Segmentation** dalam **metode pendeteksian jalur (lane) pada citra atau video** yang menggunakan pendekatan **segmentasi instance** untuk memisahkan setiap jalur secara individu. Teknik ini banyak digunakan dalam **sistem kendaraan otonom (autonomous driving)** dan **driver assistance system (ADAS)** untuk membantu kendaraan mengenali batas lajur di jalan. Memungkinkan **deteksi jalur secara lebih akurat** dengan membedakan jalur dari lingkungan sekitar.

Berikut penjelasan konsepnya:

1. **Lane Detection (Deteksi Jalur)**

Tujuan utama dari lane detection adalah **mengenali dan melacak posisi marka jalan** (garis putih/kuning yang menandai lajur kendaraan) dari citra kamera kendaraan. Hasil deteksi ini digunakan untuk menjaga agar kendaraan tetap berada di jalur yang benar, membantu fitur seperti **lane keeping assist** atau **autonomous navigation**.

2. **Instance Segmentation (Segmentasi Instance)**

Instance segmentation adalah **gabungan antara deteksi objek dan segmentasi piksel**. Metode ini tidak hanya mengklasifikasikan area mana yang termasuk jalur, tetapi juga **memisahkan setiap jalur sebagai objek unik** (instance berbeda). Dengan demikian, setiap garis jalan kiri, tengah, dan kanan dapat dikenali secara individual.

3. Cara Kerja Umum

- **Input:** citra atau video dari kamera depan kendaraan.
- **Preprocessing:** dilakukan normalisasi dan peningkatan kontras agar marka jalan lebih terlihat.
- **Feature Extraction:** menggunakan model CNN (Convolutional Neural Network) untuk mengenali pola khas dari marka jalan.
- **Segmentation:** sistem menghasilkan peta piksel yang menunjukkan area jalur.
- **Instance Separation:** setiap jalur dipisahkan menjadi instance terpisah menggunakan teknik seperti **Mask R-CNN** atau **LaneNet**.

4. Keunggulan Pendekatan Ini:

- Dapat **membedakan setiap jalur secara terpisah**, bukan hanya mendeteksi area jalan secara umum.
- Lebih **akurat pada kondisi kompleks**, seperti jalan bercabang, tikungan tajam, atau marka jalan ganda.
- Mampu beradaptasi terhadap **berbagai kondisi pencahayaan dan cuaca** jika dilatih dengan dataset yang luas.

4. Analisis dan Diskusi

- **Analisis**

Deteksi jalur rel kereta menggunakan Instance Segmentation (YOLO) dan metode klasik Canny Edge Detection menawarkan kekuatan yang saling melengkapi. Instance Segmentation unggul dalam robustness dan akurasi semantik karena, sebagai model *deep learning*, ia mampu secara spesifik mengidentifikasi objek "rel" terlepas dari perubahan pencahayaan, bayangan, atau tekstur latar belakang (kerikil/rumput), menghasilkan *mask* poligon yang presisi. Sebaliknya, Canny Edge Detection adalah algoritma cepat berbasis gradien yang mendeteksi semua perubahan intensitas yang signifikan dalam citra. Secara terpisah, Canny rentan terhadap *noise* visual, menghasilkan banyak *false positives* dari kerikil atau bayangan, sehingga deteksi jalurnya kurang andal tanpa pemrosesan lebih lanjut (seperti Hough Transform).

Kombinasi kedua metode secara signifikan meningkatkan akurasi dan keandalan. YOLO menyediakan Regional-of-Interest (ROI) semantik (area biru pada *mask*), yang kemudian menjadi *filter* bagi hasil Canny. Tepi Canny yang terletak di dalam *mask* YOLO dapat dipercaya sebagai tepi rel yang sebenarnya, memberikan akurasi piksel yang tinggi untuk memperhalus kontur rel, sementara tepi di luar area tersebut dapat diabaikan. Dampak dari perubahan parameter Canny, sangat krusial; jika ambang batas diatur terlalu rendah, Canny akan mendeteksi terlalu banyak *noise* yang mengganggu; jika terlalu tinggi, tepi rel yang sebenarnya, terutama di area bayangan atau kontras rendah, dapat terputus-putus.

- **Diskusi**

1. Seberapa baik deteksi jalur dengan Instance Segmentation dibandingkan Canny?

Instance Segmentation	Canny
1. Model dilatih untuk mengenali fitur spesifik rel (bentuk, pola <i>sleeper</i>)	1. Sangat Sensitif terhadap <i>noise</i> , bayangan, dan kondisi pencahayaan yang berubah-

kayu/beton, tekstur baja) dan dapat mengabaikan <i>noise</i> lingkungan (bayangan, batu kerikil, rumput). Ini menghasilkan deteksi yang stabil meskipun ada perubahan drastik dalam pencahayaan atau cuaca. 2. Outputnya adalah mask poligon dan label kelas ("rail-detection"), yang secara eksplisit memisahkan objek rel dari latar belakang. Ini ideal untuk aplikasi yang memerlukan pengukuran area atau analisis bentuk. 3. Karena telah mempelajari konteks dan bentuk keseluruhan objek, model tidak mudah tertipu oleh diskontinuitas lokal yang tidak relevan.	ubah. Sulit untuk membedakan antara tepi rel dan tepi batu di sekitarnya. 2. Citra Biner Tepi (hanya piksel putih) yang menandai semua perubahan intensitas yang signifikan. 3. Kurang Baik secara independen karena mendeteksi terlalu banyak <i>noise (false positives)</i> di sekitar rel (batu, rumput, bayangan).
--	---

2. Apakah kombinasi kedua metode dapat meningkatkan akurasi?
Ya, segmentasi YOLO memberi tahu di mana letak rel. Tepi Canny yang berada **di dalam** area mask YOLO memiliki probabilitas sangat tinggi sebagai tepi rel yang sebenarnya. Tepi Canny **di luar** mask dapat diabaikan (*noise*). Tepi yang dihasilkan oleh Canny (yang memiliki akurasi piksel yang tinggi) dapat digunakan untuk memperhalus kontur mask segmentasi YOLO. Jika YOLO gagal mendeteksi rel (misalnya karena rel tertutup lumpur), tetapi algoritma Hough Line Transform yang diterapkan pada tepi Canny di **ROI rel** masih mendeteksi garis lurus, ini bisa menjadi mekanisme *fallback* atau peringatan anomali.
3. Apa dampak perubahan parameter Canny (thresholds) terhadap hasil deteksi?
 - Ambang Batas Terlalu Rendah (Low = 30, High = 90): Mendeteksi terlalu banyak *noise* (batu, kerikil, rumput,

bayangan) selain tepi rel, akan menemukan garis palsu (false positives) dari *noise* tersebut.

- Ambang Batas Terlalu Tinggi (Low = 150, High = 250): Hanya tepi dengan kontras tertinggi yang terdeteksi. Rel mungkin terdeteksi dengan baik di bawah sinar matahari langsung, tetapi tepi rel mungkin **terputus-putus** di area bayangan atau saat kondisi pencahayaan meredup, sehingga sulit bagi algoritma pelurusan garis (seperti Hough) untuk bekerja.
- Pengaturan Ideal (Low = 50, High = 150): mendeteksi tepi rel dengan baik sambil memfilter sebagian besar *noise* latar belakang.

5. Assignment

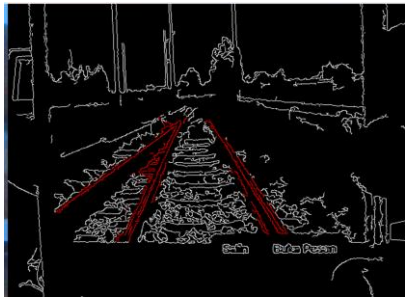
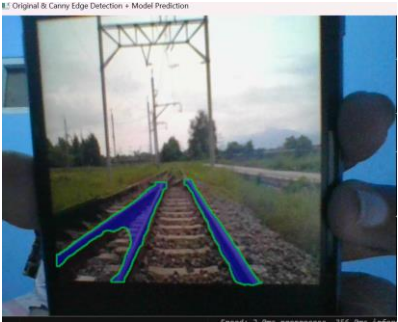
Pada praktikum ini dilakukan penerapan Canny Edge Detection & Lane Detection with Instance Segmentation. Output Canny Edge Detection divalidasi oleh *mask* YOLO, menghasilkan visualisasi tepi yang selektif. Pada hasil Canny murni (tepi putih), terlihat kepadatan tepi yang sangat tinggi pada area ballast (kerikil), yang merupakan *noise* dari sudut pandang deteksi jalur. Melalui proses validasi silang (di mana hanya tepi di bawah *mask* biru yang dipertahankan), *noise* ini secara efektif tereliminasi. Tepi Canny yang divalidasi dan berubah menjadi merah secara eksklusif berlokasi pada batas-batas rel yang tersegmentasi. Hal ini mengkonfirmasi bahwa Instance Segmentation YOLO berhasil mengidentifikasi Region of Interest (ROI) yang paling relevan, memungkinkan algoritma Canny (dengan akurasi piksel yang tinggi) untuk mengisolasi batas rel secara tepat

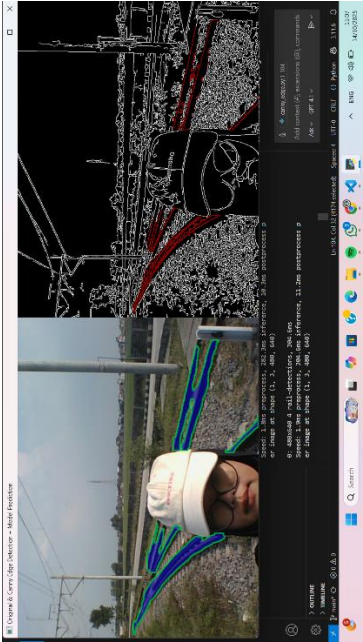
Berdasarkan *output* visualisasi sistem hibrida (Gambar 2/3/4), kombinasi Instance Segmentation (YOLO) dan Canny Edge Detection menunjukkan kinerja yang efektif. Model YOLO yang dilatih **best.pt** berhasil melakukan segmentasi instans pada jalur rel.

Model menunjukkan *robustness* yang tinggi dengan tetap mengidentifikasi rel meskipun terjadi oklusi signifikan oleh subjek di latar depan. *Mask* **biru** transparan secara akurat memetakan piksel rel yang terlihat, membuktikan

kemampuan model untuk **memahami konteks semantik** dari objek 'rel' daripada hanya mencari pola geometris sederhana.

6. Data dan Output Hasil Pengamatan

No	Keterangan	Output
1	Hasil dari Canny Edge Detection Citra ini menampilkan representasi tepi (garis putih) dari objek-objek dalam bidikan rel kereta api dengan latar belakang hitam pekat. Mengisolasi semua diskontinuitas intensitas yang signifikan dalam gambar asli.	
2	Hasil dari Combined Detection Menampilkan hasil <i>real-time</i> dari sistem deteksi hibrida yang menggabungkan YOLO Instance Segmentation dan Canny Edge Detection. Layar terbagi menjadi dua panel utama.	

3	Hasil dari Combined Detection Instance Segmentation (biru) memberikan pemahaman semantik di mana rel berada, sementara Canny (putih/merah) memberikan presisi piksel dari tepi tersebut. Kombinasi ini mengatasi kelemahan masing-masing metode, menghasilkan output yang <i>robust</i> dan informatif.	
4	Bagian jalur rel yang terdeteksi ditutupi oleh mask biru pekat dan transparan. Ini adalah output dari Instance Segmentation, di mana model telah berhasil mengidentifikasi dan menandai piksel yang termasuk dalam objek "rail" Setiap bagian rel yang terdeteksi ditandai dengan kotak biru yang mengelilingi area segmen tersebut (<i>bounding box</i>).	

7. Kesimpulan

Berdasarkan percobaan yang dilakukan dapat disimpulkan bahwa:

1. Berhasil mempraktikkan konsep Canny Edge Detection & Lane Detection with Instance Segmentation.
2. Model Instance Segmentation (YOLOv8, best.pt) menunjukkan kemampuan semantik yang unggul dalam mendeteksi dan mensegmentasi jalur rel secara akurat (*mask biru*), bahkan ketika terdapat oklusi atau variasi pencahayaan yang tinggi.

3. Canny Edge Detection tidak berfungsi sebagai detektor jalur independen, melainkan sebagai validator presisi tinggi, mengkonfirmasi dan memperhalus batas piksel yang teridentifikasi oleh YOLO.
4. Kombinasi kedua metode menciptakan sistem yang robust dan akurat.
5. Implementasi berhasil membedakan dan menyajikan informasi dari kedua metode (segmentasi biru, tepi Canny putih, dan tepi terkonfirmasi merah) bersama dengan label deteksi profesional

8. Saran

Untuk memperkuat pemahaman konsep dan implementasi, disarankan agar praktikum berikutnya melakukan integrasi pasca-pemrosesan di mana tepi Canny yang divalidasi (garis merah) digunakan untuk mengukur parameter geometris rel, seperti lebar jalur (gauge) atau pelacakan titik hilang, yang tidak dapat diberikan oleh *mask* YOLO secara langsung. Melakukan analisis sensitivitas parameter Canny terhadap *inference speed* dan akurasi di lingkungan yang berbeda (malam hari, kabut) untuk menemukan ambang batas yang paling optimal.

9. Daftar Pustaka

- Ultralytics. (2023). *YOLOv8: YOLO by Ultralytics* (Version 8.0). <https://github.com/ultralytics/ultralytics>
- OpenCV Development Team. (2024). *OpenCV (Open Source Computer Vision Library)* <https://opencv.org/>